דף סיכום בחינה

[redacted]

מזהה קורס: 20594   שם קורס: מערכות הפעלה

מספר שאלה	ניקוד מירבי	ציון
1	26.00	26.00
2	17.00	17.00
3	12.00	12.00
4	5.00	5.00
5	5.00	5.00
6	5.00	4.00
7	5.00	5.00
8	5.00	0.00
9	5.00	5.00
10	5.00	5.00
11	5.00	5.00
12	5.00	5.00

ציון בחינה סופי : 94.00

הבחינה הבדוקה בעמודים הבאים

[redacted]

[redacted]

מספר התלמיד הנבחן
רשום את כל תשע הספרות

האוניברסיטה הפתוחה

ט' באדר תשפ"ג

2

מס' שאלון - 516

במרץ 2023

סמסטר 2023א

מס' מועד 92

20594 / 4

שאלון בחינת גמר

20594 - מערכות הפעלה

משך בחינה: 3 שעות

בשאלון זה **10** עמודים

מבנה הבחינה:

קראו בעיון לפני שתתחילו בפתרון הבחינה!

א. המבחן מורכב משלושה חלקים.

ב. בחלקים א ו - ב מופיעות שאלות פתוחות. ענו תשובות מלאות, בכתב קריא ובקיצור נמרץ. אין חובה להשתמש בכל השורות המוקצות לצורך התשובות, אך אין לחרוג מהמקום המוקצה.

ג. בחלק ג (שאלות אמריקאיות) עליכם לבחור בכל פעם בתשובה יחידה מבין התשובות המוצעות ולהקיף בעיגול את אות התשובה שבחרתם.

את התשובות לכל השאלות יש לכתוב בשאלון הבחינה.

חומר עזר:
כל חומר עזר אסור בשימוש, פרט למחשבון, שאינו אוצר מידע.

בהצלחה !!!

החזירו
למשגיח את השאלון
וכל עזר אחר שקיבלתם בתוך מחברת התשובות

שאלון 516

92.89.2 M3

חלק א - 55 נקודות

שאלה 1 (26 נקודות)

נתונה בעיה של יצרן אחד ושלושה צרכנים שבה ישנם שלושה מוצרים שונים המסופקים על ידי היצרן וכל אחד מן הצרכנים נזקק לשניים מהם. יצרן מקבל כל כמה זמן אספקה של 2 מהמוצרים (רנדומאליים, אבל לא זהים) ומציע אותם לצרכנים. הצרכן, שאלו שני המוצרים הדרושים לו, לוקח(מנסה) את שני המוצרים. רק לאחר שהמוצרים נלקחו, חוזר היצרן על התהליך - ומציע שוב שני מוצרים(מתי שהם זמינים). כך חוזר התהליך על עצמו שוב ושוב. נדרש פתרון לבעיה המשתמש אך ורק בסמפורים כאמצעי סנכרון .

לבעיה הוצע הפיתרון הבא, המשתמש במערך [S[0..2 של סמפורים ובסמפור נוסף בשם producer (אין זה משנה אם הסמפורים בינאריים או כלליים) המאותחלים ל 0 . בפתרון זה כל אחד מן הסמפורים S [i] מיצג אחד מן המוצרים.

Producer pseudo code , i and j are two random not equal products from { 0 , 1 , 2 }	Consumer pseudo code , i ∈ { 0 , 1 , 2 }

```
producer() {
    1. while( true) {
    2. GetRandoms( i , j , { 0 , 1 , 2 } ) // i != j
    3. Up( S[ i ] ) // "Place" product i
    4. Up( S[ j ] ) // "Place" product j
    5. Down( producer )
    6. }
}
```

```
consumer( i ) {
    1. while( true ) {
    2. Down( S[ i ] ) // "Take" product i
    3. Down ( S[ ( i + 1 ) mod 3] )
    4. Up( producer )
    5. }
}
```

א. (6 נק') תנו תסריט המוכיח כי בפתרון זה יתכן קיפאון deadlock

נבחן את התרחיש הבא: לקוח 0 מעוניין במוצרים 0,1 לקוח 1 ב 1,2 ולקוח 2 במוצרים 0,2. לקוח 0 מספיק לבצע את שורה 2 לכן ממתין למוצר 0, יש CSW לקוח 1 מספיק לבצע את שורה 2 לכן ממתין למוצר 1, יש CSW לקוח 2 מספיק לבצע את שורה 2 לכן ממתין למוצר 2. יש CSW לקוח 0 מבצע את שורה 3 וכעת ממתין למוצר 1. יש CSW לקוח 1 מבצע את שורה 3 וממתין למוצר 2. יש CSW לקוח 2 מבצע את שורה 3 וממתין למוצר 0. התלויות של מוצר 0: לקוח 0 ואז לקוח 2. התלויות של מוצר 1: לקוח 1 ואז לקוח 0. התלויות של מוצר 2: לקוח 2 ואז לקוח 1.
כיוון שהמפיק יקבל מוצרים למלאי ויעיר את הצרכנים לא משנה מי מהם יתעורר קיבלנו מעגל המתנה שהוביל לקיפאון. (אחרי שיהיו 3 מוצרים כל אחד יחזיק במוצר וירצה מוצר אחר שמישהו אחר כבר מחזיק 12.)

שאלון 516

92.89.2 M3

ב. (15 נק') הציעו קוד חלופי הפותר בעיה זו ומשתמש בדיוק באותם משתנים (אך משנה את משמעותם). ניתן להניח כי היצרן יודע מי הם הצרכנים ולכן גם יודע אילו מוצרים כל אחד מהם צריך.

מותר להוסיף פקודות(בפסודוקוד), אבל אסור להוסיף משתנים נוספים.

עבור 3 מוצרים 0,1,2 יש בדיוק 3 צירופים אפשריים שהצרכן יכול לקבל אותם: {0,1}, {0,2}, {1,2}. כמובן, הצרכן יודע לכל לקוח אילו מוצרים הוא רוצה. לקוח 0 - מוצרים 0,1, לקוח 1 מוצרים 1,2, לקוח 2: מוצרים 2,0. כאשר S[i] "יעיר" לקוח i ולא מוצר i. היצרן יעביר פעם צרכן בהתאם ל-2 המוצרים הרנדומליים שקיבל. אלו הם בדיוק 2 המוצרים שאותו צרכן מחכה להם.

```
consumer(i) {                    producer () {
  while (true) {                   while (true) {
    down(S[i]);                      Get Randoms (i,j, {0,1,2})
    up(producer);                    if (i,j ∈ {0,1}) {
  }                                    up S[0]
}                                    }
                                     if (i,j ∈ {0,2}) {
                                       up S[2]
                                     }
                                     if (i,j) ∈ {1,2}) { up S[1] }
                                     down (producer);
                                 } }
```

הפכנו את הדרישה של כל לקוח לפעולה אטומית

26
(1)

ג. (5 נק') האם תיתכן הרעבת חלק מצרכנים בקוד שבסעיף א' ? הוכיחו שלא או תנו דוגמא שכן.

תשובה והסבר

בגלל האקראיות באופן קבלת המוצרים בצד היצרן הסיכויים להתעכב צרכן מסוים נמוכה מאחר ובכל פעם בשורה 2 של היצרן הוא מקבל למעשה 2 מוצרים באופן רנדומלי מתוך 3 מוצרים ומציע אותם לצרכנים בהתאם לסוג המוצר שקיבל - מעיר את הצרכן הרלוונטי ומציע לו אותו. לכן במצב כזה לא תיתכן הרעבת חלק מהצרכנים (הודות לרנדומליות).

שאלה 2 (17 נקודות)

נתון זיכרון פיזי (RAM) בגודל של 32MB (קטן בשביל פשטות החישובים).

המהנדסים החליטו לבנות את מערכת הזיכרון עם מסגרות (frames) בגדלים שונים.

הדפים/המסגרות יהיו בגדלים הבאים: 2k, 4k, 8k ו 16k. הרבע הראשון של הזיכרון יחולק לגודל של 2k, הרבע השני יחולק לגודל של 4k, הרבע השלישי יחולק לגודל של 8k והרבע הרביעי יחולק לגודל של 16k.

$2k = 2^1 \cdot 2^{10} B = 2^{11} B$
$4k = 2^2 \cdot 2^{10} B = 2^{12} B$
$8k = 2^3 \cdot 2^{10} B = 2^{13} B$
$16k = 2^4 \cdot 2^{10} B = 2^{14} B$

א. (7 נק')

נתונות שלוש כתובות ווירטואליות VIRTUAL ADDRESS הכתובת הבאות(בייצוג בינארי):

.1 23

1	0	1	0	0	0	0	0	0	0	1	1	1	1	1	0	0	0	0	0	0	0	1

~~.2~~ 24

1	0	1	0	0	0	0	0	0	0	1	1	1	1	1	0	0	0	0	0	0	0	1	1

.3 25

1	0	1	0	0	0	0	0	0	0	1	1	1	1	1	0	0	0	0	0	0	0	0	1	1

רביע של הזכרון — מס' דף — offset

17
(2)

מה היא צורת הכתובה הנכונה במערכת הנ"ל?(1 ,2 או 3). הסבירו את תשובתכם.

מה גודל ומספר המסגרת? מהו ה OFFSET שבכתובת הנכונה מבין שלוש?

צורת הכתובה הנכונה היא 3 = סה"כ 25 ביטים לכתובת. $32MB = 2^{25}B$

הכתובת בנויה כך - 2 הביטים הכי שמאליים מייצגים את הרביע בזכרון - 00 (1), 01 (2), 10 (3), 11 (4).

בהתאם לרביע שאנחנו נמצאים בו ה offset יהיה: 11 סיביות לרביע 1, 12 לרביע השני,

13 לרביע השלישי ו- 14 לרביע האחרון. מס' הדף יהיה בהתאם לסיביות הנותרו מתוך ה 25 שלו

ביטים שמאליהם. לגבי ה Offset בכתובת הנכונה מבין ה-3: נסתכל על הסיביות

שמאליות. אנחנו ברביע השלישי, ברביע זה גודל דף 8K לכן ה offset בגודל 13 ביטים

ואלה הביטים הבאים:

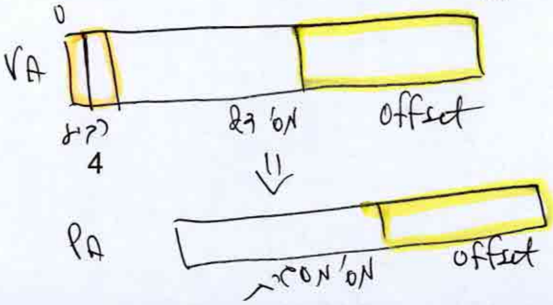
1	1	1	0	0	0	0	0	0	0	0	1	1

ב. (10 נק')

תארו את אלגוריתם פשוט לתרגום כתובת ווירטואלית VIRTUAL ADDRESS לכתובת פיזית שיאפשר ל MMU לבצע את התרגום ביעילות במערכת הנ"ל.

האלגוריתם יעבוד כך: ראשית יסתכל על 2 הביטים הכי שמאליים מתוך 25 הביטים
של ה VA ובהתאם אליהם ידע את גודל ה offset. 00 - offset בגודל 11
01 - offset בגודל 12, 10 - offset בגודל 13, 11 - offset בגודל 14.
כעת אנחנו יודעים מהו ה offset וגם מהו ה offset בכתובת הפיזית כי גודל דף = גודל מסגרת.
כעת כל שנותר הוא להסתכל ב TLB / בטבלת הדפים (אם לא מצאנו ב TLB)
ולמצוא עבור כל דף מהי המסגרת אליה הוא ממופה (מס' מסגרת). לבסוף
PA מורכבת מה offset שאנחנו יודעים כי הוא זהה לזה ב VA ולמס'
מסגרת שלקחנו מטבלת הדפים.

המשך הבחינה בעמוד הבא



שאלה 3 (12 נקודות)

במערכת הפעלה מסוימת מערכת קבצים של דיסק קשיח מבוססת I- NODES .

א. (3 נקודות)

האם מבחינת זמן גישה יש יתרון להקצאה רציפה של בלוקים של הנתונים לקובץ? בתשובה התייחסו לגישה רציפה ולגישה אקראית.

תשובה והסבר

עבור גישה רציפה - יש יתרון כיון שצריך לבצע פחות הזזות של זרוע הדיסק מאחר ובלוקי המידע קרובים אחד לשני.

עבור גישה אקראית - עדיין יש יתרון כי עדיין נצטרך לבצע פחות הזזות של זרוע הדיסק מאשר אם בלוקי הנתונים לא היו מוקצים בצורה רציפה אחד ליד השני.

ב. (2 נקודות)

האם יש יתרון להקצאת בלוקים עם המצביעים הלא ישירים בסמוך לבלוקים של נתונים של הקובץ?

תשובה והסבר

כן, יש יתרון להקצאת בלוקים עם המצביעים הלא ישירים בסמוך לבלוקים של נתונים של הקובץ, כאשר הם מוקצים קרוב אחד לשני זרוע הדיסק תצטרך לזוז פחות מאשר אם הם היו רחוקים וזה יכול לשפר את מהירות הקריאה/כתיבה של הנתונים.

12
(3)

ג. (4 נקודות)

בלי קשר לתשובה לסעיף ב', במערכת שבא כל הבלוקים עם המצביעים של כל הקבצים נשמרים בתחילתו של הדיסק הקשיח ובזמן יצירת קובץ חדש יוקצה לו מקום החל מהאמצע של הקטע הפנוי הארוך ביותר שקיים במערכת. מה הן היתרונות והחסרונות של השיטה הזאות?

תשובה והסבר

חסרונות - שיטה זו יכולה לגרום לריסוק חיצוני גדול בגלל אופן ההקצאה של השטחים לקבצים חדשים. בנוסף, יכול להיות שנגיע למצב בו נהיה חייבים להקצות לקובץ חדש שטח שיועד להיות מקום להתרחבות של קובץ אחר וכך נאבד את היתרון של השיטה (כשאין הרבה מקום פנוי להקצאה).

יתרונות - מאפשר לקבצים להגדיל את שטחם בזכרון באופן רציף, גישה יותר מהירה אליהם עקב כך. כשרוצים להקצות קובץ עוד בזכרון, אם מקום פנוי זה חוסך זמן חיפוש בשאר הזכרון ולכן ההקצאה מהירה יותר.

ד. (3 נקודות)

כשמדובר באותה שיטה כמו בסעיף ג', באיזו צורה(רשימה משורשרת ממוינת או מפת סיביות) עדיף לשמור את הנתונים לגבי מקום פנוי במערכת קבצים?

תשובה והסבר

עדיף לשמור את הנתונים באמצעות רשימה משורשרת ממוינת מאחר והיא מאפשרת לנו לחפש באופן יותר מהיר שטחים פנויים ולהקצות אותם. בנוסף כשבלוק מוקצה פשוט נמחק אותו מהרשימה בצורה קלה.
במפת סיביות החיפוש אחר שטח פנוי יותר ארוך ומסורבל כי יש לחפש רצף באורך מתאים וזה לוקח יותר זמן.

חלק ב' - 15 נקודות, 5 נקודות לכל שאלה

שאלה 4

א. למה בממ"ן 11 (הוספת פקודת מרחב המשתמש שמדפיסה פרטים על התהליכים הרצים במערכת) היה צורך להוסיף גם SYSTEM CALL ולא היה מספיק להוסיף פקודת מרחב המשתמש ולממש בה את הפעולות הדרושות?

ב. מה תפקידה של פקודת מרחב המשתמש(מה היה קורה אם היו מוסיפים רק את SYSTEM CALL) ובשביל מה צריך להוסיף אותה?

5
(4)

תשובה והסבר

פקודת מרחב משתמש מוגדרת בצורת API שאומר אילו קריאות המשתמש יכול לבצע אל מול מערכת ההפעלה כך שניתן לו משאבים מסוימים שהוא מעוניין בהם, כדי להשתמש במשאבים אלה חייבים לעבור למצב מיוחס ולכן חייבים לממש את הפקודה גם במצב מיוחס ולא מספיק רק במרחב המשתמש.
צריך להוסיף את ה SYSTEM CALL כדי שה CPU ידע מה לעשות כשקורא מבקש ממנו פעולה מסוימת, לכל קריאה int שמייצג אותה ונשמר לאוגר eax. לפי מס' זה מע"ה יודעת איזו פעולה לבצע ובאילו משאבים עליה להשתמש.

שאלה 5

אם במערכת הפעלה קיים SYSTEM CALL דומה למה שנוסף בממ"ן 12(המחזיר את כל המזהים PID – ים שיש לכל תהליך), איך בעזרתו תהליך משתמש יכול לגלות האם הוא רץ בתוך הקונטיינר או מחוץ לקונטיינר?

5
(5)

תשובה והסבר:

כאשר משתמשים בפקודת ps נסתכל על extpid ועל pid. ברגע שהם לא שווים אחד לשני המשמעות היא שהתהליך נמצא בתוך קונטיינר ולכן אבא שלו רואה אותו עם מזהה (extpid) שונה מאיך שהוא רואה את עצמו בתוך הקונטיינר עם ה pid.

נא לא לכתוב בשוליים

שאלה 6

מה יודפס בעקבות הרצת קטע קוד הבא בתוך מערכת xv6 שתומכת בקונטיינרים, ניתן להניח שכל קריאות מערכת מצליחות:

```
unshare (MOUNT_NS);
int pid = fork();
if (pid == 0) {
  mkdir ("dirS");
  mount ("internal_fs_a","dirS",0); //mount a preformatted internal_fs_a
  int fd = open ("dirS/file.txt",O_CREATE|O_RDWR);
  write (fd ,"123456789\n",10);
  close (fd);
 }
if (pid > 0) sleep (10000); // wait for child to finish
sleep (10000); // both processes will sleep
if( pid == 0) {
   if (open ("dirS/file.txt",O_RDONLY) < 0) printf (stderr, "son failed to open\n");
   else printf (stderr, "son succeeded\n");
 }
If (pid>0) {
   if (open ("dirS/file.txt",O_RDONLY) < 0) printf (stderr, "father failed to open\n");
   else printf (stderr, "father succeeded\n");
 }
```

4
(6)

הפרדה צריכה להיות לפני MOUNT

תשובה והסבר:

unshare(MOUNT_NS) מתבצע לפני ה fork, לכן גם האב וגם הבן
באותו namespace mount. גם האב וגם הבן יכולים לגשת אל הקובץ החדש
שנוצר ע"י הבן ולכן יודפס: "son succeeded"
"father succeeded"

המשך הבחינה בעמוד הבא

חלק ג (30 נקודות)

כשתשובה לא נכונה, הציון מופיע לצד הנכונה

ענו על שש שאלות רב-ברירה (אמריקאיות). משקל כל שאלה 5 נקודות.

בכל שאלה יש לבחור את התשובה הנכונה ביותר ולהקיף בעיגול את אות התשובה שבחרתם.

שאלה 7

בניסיון למנוע את מצב הקיפאון DEADLOCK במערכת נקבע הפרוטוקול הבא:

a. לכל משאבי המערכת יש מספרים סידוריים.

b. כל תהליך יכול לבקש הקצאת משאבי המערכת רק בסדר עולה של המספרים ולשחרר אותם אחרי גמר השימוש רק בסדר יורד.

5
(7)

האם הפרוטוקול הזה אכן מונע את אפשרות הקיפאון?

[✓ א.] כן, כי לא יכולה להיווצר המתנה מעגלית למשאבים.

ב. כן, אבל רק אם כל תהליך משחרר את כל המשאבים שכבר קיבל אם הוא לא מצליח לקבל משאב נוסף.

~~ג.~~ לא, אם למשל במצב הנתון יש במערכת 3 תהליכים, ראשון קיבל משאב 1 וכדי להמשיך צריך משאב 3 שמוחזק ע"י תהליך שני שמשתמש בו.

~~ד.~~ לא, אם למשל במצב הנתון יש במערכת 2 תהליכים, ראשון קיבל כעת משאב 1 וכדי להמשיך צריך משאב 3 שמוחזק ע"י תהליך שני שצריך משאב 2 כדי להמשיך ובעת הוא מוחזק ע"י תהליך 1 .

ניתן להוכיח שכן כי לא יכולה להיווצר המתנה מעגלית למשאבים.

שאלה 8

האם בארכיטקטורות "ישנות" שאינן תומכות בווירטואליזציה ישירה(VT) ניתן לממש בכל זאת ווירטואליזציה מלאה מבוססת רב-פקח HYPERVISOR שמתרגם קוד בזמן ריצה: כל פעם שקטע קוד חדש רץ במכונה הווירטואלית , הוא "יתוקן" על ידי הרב-פקח כך שפקודות בעייתיות(רגישות) יוחלפו .

וירטואליזציה
מלאה =
וירטואליזציה
חומרה.

כיצד יכול הרב-פקח לדעת ולהתערב כל פעם שקוד חדש רץ, בלי להאט משמעותית ריצה של קוד אשר כבר תוקן (אין צורך להסביר איך הקוד יתוקן)?

א. ניתן לממש בעזרת פסיקות דף ע"י הגדרת הדפים עם הקוד שעדיין לא התבצע כלא ניתנים להרצה בטבלת הדפים של המכונה המארחת HOST

0
(8)

ב. ניתן לממש בעזרת פסיקות דף ע"י הגדרת הדפים עם הקוד שעדיין לא התבצע כלא ניתנים להרצה בטבלת הדפים של המכונה האורחת GUEST .

[✓ ג.] לא ניתן לממש כי אין מנגנון שיקרא ל KERNEL המארח בזמן ביצוע פקודות רגישות במכונה אורחת.

~~ד.~~ ניתן לממש בעזרת הרצת המכונה האורחת במצב אמיתי REAL MODE .

שאלון 516

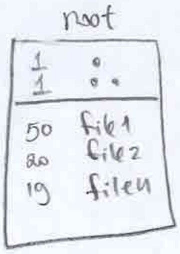
92.89.2 M3

נא לא לכתוב בשוליים

שאלה 9

מה המידע המצוי בקובץ ספרייה (Directory) של מערכת הקבצים במערכת LINUX(מבוססת I-NODE) ?

5
(9)



א. שמות הקבצים ולכל קובץ מספר הבלוק הראשון שלו.

[✓ ב. שמות הקבצים ולכל קובץ מספר ה-i-node שלו.]

ג. שמות הקבצים ולכל קובץ התכונות (Attributes) שלו.

ד. שמות בקבצים ולכל קובץ רשימת הבלוקים שהוקצו לו.

שאלה 10

כאשר מדובר על מבנה מערכת הפעלה לפי מודל שרת-לקוח (client-server model) מהי התכונה אשר מהווה חסרון מובהק של המודל?

א. תקורת (overhead) התקשורת בין תהליכי המשתמש.

ב. חוסר אפשרות התאמה למערכות מבוזרות (distributed systems).

[✓ ג. תקורת (overhead) התקשורת בין רכיבי המערכת.]

ד. המבנה מתאים רק מערכות רשתות תקשורת.

5
(10)

שאלה 11

אם משתמשים ב- first fit להקצאת זיכרון RAM רציף, **לא ייתכן** שלקטע האחרון שהוקצה תהיה התכונה הבאה:

[✓ א. יש קטע שאינו מוקצה/תפוס גדול יותר ממנו בכתובת נמוכה יותר.]

ב. יש קטע שאינו מוקצה/תפוס קטן יותר ממנו בכתובת גבוהה יותר.

ג. יש קטע שאינו מוקצה/תפוס צמוד אליו בכתובת נמוכה יותר.

ד. יש קטע שאינו מוקצה/תפוס קטן יותר מהסכום של הגודל שלו והגודל של קטע בלתי-מוקצה שכן.

5
(11)

ב-FF עוברים על רשימה של שטחי הזכרון הפנויים באופן רציף מהכתובת הנמוכה לגבוהה כל פעם.
ברגע שמוצאים זכרון גדול דיו להכיל את הדרישה שלנו בוחרים בו ומקצים ממנו רק את השטח הדרוש לנו (את השאר משאירים פנוי).
לכן לא יכול להיות שיש קטע שאינו מוקצה שגדול יותר מהקטע האחרון שהוקצה שנמצא בכתובת נמוכה יותר.

המשך הבחינה בעמוד הבא

כאן לא לכתוב בשוליים

שאלה 12

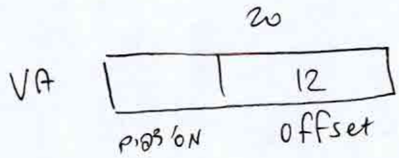
5
(12)

נתון כי במערכת מסוימת המשתמשת בשיטת הדפדוף (paging) לצורך ניהול הזיכרון:

- גודל הדף הוא 4 K.
- הכתובת הוירטואלית הנה בת 20 סיביות (bits).
- גודל הזיכרון הפיזי הוא 640 KB.
- גודל שורה בטבלת הדפים (page table entry) הוא 16 בתים (bytes).

אם טבלת הדפים הנה **טבלת הדפים המהופכת** (inverted page table), מהו גודל הטבלה?

[✓ א.] 640 בתים.
ב. 1024 בתים.
ג. 2048 בתים.
[✓ ד.] 2560 בתים.



$4KB = 2^2 \cdot 2^{10}B = 2^{12}B$

בהצלחה!

$e \quad 16B = 2^4 B$

$\frac{640KB}{4KB} = 160$

נא לא לכתוב בשוליים